



RECO-ASCON: Reconfigurable ASCON Hash Functions for IoT Applications

Mohamed El-Hadedy^{a,e,*}, Xinfei Guo^{b,e,*}, Kazutomo Yoshii^c, Yichen Cai^b, Robert Herndon^a, Bryan Banta^a, Wen-Mei Hwu^d

^aDepartment of Electrical and Computer Engineering, California State Polytechnic University, Pomona, California, USA

^bUniversity of Michigan – Shanghai Jiao Tong University Joint Institute, Shanghai Jiao Tong University, Shanghai, China

^cArgonne National Laboratory, Lemont, Illinois, USA

^dUniversity of Illinois at Urbana-Champaign, Illinois, USA

^eThese authors contributed equally to this work.

Abstract

The need for reconfigurable lightweight cryptographic processors with low power consumption for securing the Internet of Things (IoT) with adaptive functionalities is getting attracted due to security properties, such as data fidelity, provenance, and privacy concerns. This paper presents RECO-ASCON, a reconfigurable security processor that supports both cryptographic Hash and Hasha ASCON algorithms. The proposed design is implemented in Chisel hardware construction language, an open-source hardware library. It has been evaluated in various hardware platforms such as FPGAs, embedded systems and ASICs with various technology nodes and design flows. In the PYNQ-Z1 board, RECO-ASCON can be mapped to five processors in total with a Micro-Blaze soft-core processor for streaming data to all of them while running at 200 MHz without producing any timing violations. Moreover, mapping the proposed RECO-ASCON processor on the Nexsys4-DDR Xilinx Kit (ARTIX-7 chip) would cost $\sim$ two times fewer resources and run $1.25 \times$ faster than the existing implementation of just one of the ASCON hash functions on the same family of the chip. We implement the proposed processor in 28nm industry technology with commercial design flow, it achieves a maximum frequency of 1.12GHz while achieving highest energy efficiency compared to the state-of-the-art implementations. The total area is only $9170.82\mu m^2$ in 28nm. To further evaluate the design, we also implement it with all open-source design tools and process technology libraries in 45nm, it achieves 1GHz maximum frequency and consumes only $18225\mu m^2$ in 45nm. Overall, the proposed reconfigurable processor is compact and more energy efficient, making it a perfect candidate to be deployed in wide range of IoT systems.

Keywords: Cryptography; IoT devices; ASIC; ASCON; Sponge-based hash functions

1. Introduction

As the Internet of Things (IoT) becomes a uniform environment with limited resources and power budget, edge computing paradigm was created by researchers to perform computations on tiny devices instead of in a centralized computing hub, making it an excellent candidate to be deployed in IoT applications [1, 2]. From understanding the requirements of designing an efficient IoT system in terms of computing capabilities, there are numerous design constraints that include cost, energy, power, and lifetime [3, 4, 5]. Historically, data privacy has been considered a lower priority in the Internet of Things (IoT). However, as the popularity of IoT systems is gaining, data privacy

*Corresponding author

Email addresses: mealy@cpp.edu (Mohamed El-Hadedy), xinfei.guo@sjtu.edu.cn (Xinfei Guo)

has become a significant concern for users [6]. As the complexity of the IoT systems increases, it is becoming extremely challenging or even impossible to detect data corruption that hardware failures, software bugs, malicious attacks, human errors, or complex combinations can induce. Thus, a lightweight hardware mechanism is worth exploring to improve data privacy and integrity for next-generation IoT systems [7, 8]. The conventional cryptographic primitives widely used in information security are not a good fit for such a constrained environment as IoT [9]. For this reason, the National Institute of Standards and Technology NIST has launched a worldwide competition in 2018 for defining the new lightweight cryptographic primitives (LWC) [10]. Several algorithms were submitted, and the finalists have been selected. These algorithms are continually tested to determine the competition winner. In the competition criteria, NIST encouraged cryptographers to submit their ideas of building lightweight authenticated encryption with associated data (AEAD) algorithms as a requirement and optionally to introduce new hash functions for tiny device applications. The first round started with 56 candidates and moved to the second round phase in mid of 2019 with only 32 candidates. Ten candidates, ASCON [11], Elephant [12], GIFT-COFB [13], Grain128-AEAD [14], ISAP [15], Photon-Beetle [16], Romulus [17], Sparkle [18], TinyJambu [19], and Xoodyak [20] were selected in the final stage of the LWC competition; several submitted both AEAD and hash function, while the rest have only submitted their AEAD algorithms [21].

The competition criteria have the side channel and fault attack resistance, cost, performance, third-party analysis, and suitability for hardware and software implementations as the backbone of defining the closest candidate to be the new standard beside the central aspect of resisting the state-of-the-art attacks [22]. Several algorithms submitted to the competition mainly rely on sponge constructions, which consume operations (add, rotate, and XORing) for computing more than memory accessibility. This makes them perfect candidates for low-power applications such as IoT. Among these, ASCON [11] and GAGE [23] are two of the LWC sponge construction-based hash functions. **In February 2023, NIST selected ASCON for new lightweight cryptography standard [24]. We find it more emerging to concentrate on this official NIST standard to serve more lightweight cryptography scenarios.**

A few research have been conducted on developing hardware solutions for LWC candidates. A hardware Application Programming Interface (API) was made especially for LWC authenticated ciphers by [25] and assisted the first implementation of SpoC-128 in [26]. [27] proposed a reconfigurable PE micro-architecture, capable of efficiently accelerating multiple LWC ciphers in the S-Box or ARX categories. While some candidates provide authenticated encryption only like GIFT-COFB and Grain-128 implemented in [28], [29], sponge-based algorithms like ASCON and GAGE [30] include both authenticated encryption and hashing functions. One aspect that has been ignored was the need for flexibly supporting various security modes and flexibility to make the hardware more adaptive to various requirements from the edge devices. This adaptivity is also required from the scalability perspectives [31]. Only certain IoT devices could afford wider data-width, while some might be constrained by the hardware resources. In this case, the design of such security engine needs to be modular and scalable. It was not just as simple as widening the bus and copying more modules, the complexity came from the microarchitecture optimizations and reconfigurability implementation from the initial design time.

As hash has been playing a major role in security due to its robust structure [32], there has been long list of hash function applications such as message digest/integrity checks [33], password verification [34], data structures (programming languages) [35], game boards [36], graphics [37], compiler operation [38], and proof-of-work in Blockchain [39]. While these work mainly focused on specific hashing and how those can be adapted in their targeted domain, this paper presents a reconfigurable ASCON processor, which runs two of ASCON hash function algorithms (Hash and Hasha) on power-efficient FPGA platform (PYNQ) [40]. We have implemented ASCON's processor using Chisel. Compared to traditional hardware design approaches, Chisel offers extra productivity and scalability [41]. The proposed ASCON processor is reconfigurable as it supports both Hash and Hasha algorithms while being compact and low power. The proposed design has been evaluated on multiple platforms such as embedded system, FPGA and ASIC. For ASIC, it is implemented with commercial and open-source design flow respectively in 28nm industry-adopted design node and 45nm open-source technology node.

The rest of the paper is organized as follows. Section 2 discusses the related work. Section 3 presents the overview of the ASCON hash algorithms. While Section 4 shows the detailed architecture of the proposed RECO-ASCON processor. Moreover, Section 5 shows the evaluation results on various hardware platforms including the implementations with ASIC flows. This section also makes the comparison against other state-of-the-art implementations. We conclude the paper in Section 6.

2. Related Work

The hash function converts a given message to a fixed digest. Historically, the first hash function standard by NIST was SHA-0 [42], which has been withdrawn and altered by SHA-1 [43]. SHA-1 was developed and designed by the national security agency (NSA) to be part of the digital signature algorithm. The output digest of the SHA-1 was 160-bit. Due to discovered weakness in SHA-1, the standard was no longer approved for most cryptographic uses since 2010. NIST has recommended removing SHA-1 from products as digital signatures and instead using SHA-2 or SHA-3. In 2001, SHA-2 was designed by NSA. SHA-2 is a set of cryptographic hash functions using the Merkle-Damgård construction [44]. SHA-2 consists of six hash functions with different digests that is 256, 224, 384, and 512 bits. SHA-256 and SHA-512 are computing hash functions with eight 32-bit and 64-bit, respectively. Later, NIST launched a competition to choose the SHA-3, which can alter the SHA-2 family in case they got broken, as with SHA-0 and SHA-1. Out of the competition, SHA-3 [45] got declared, it used sponge-construction [46] to make the hash function resist the known attacks. SHA-2 and SHA-3 were valid, and no serious attack has been identified. They were widely used on a long list of applications. However, they were expensive in terms of computing and power consumption, which made them infeasible to secure the physically-limited and energy-constrained IoT devices. For this reason, the aforementioned competition made the selection for the most efficient lightweight hashing algorithm that can potentially be adapted in the IoT scenarios.

ASCON [11] is one of these few candidates' submissions consisting of both hashes and Authenticated Encryption with Associated Data (AEAD), which depends on using Sponge construction to construct the security engine of both AEAD and hash functions. There are only few hardware implementations for ASCON, and most of these work focused on single instanced design. In [47], authors introduced a reconfigurable ASCON processor that runs all modes of the ASCON family from AEAD to hash functions. But the focus was on optimizing the design as a whole instead of only hashing functions. In addition, that work didn't offer considerations of how the processor can be integrated on a system on chip (SoC). In this paper, the proposed processor named RECO-ASCON is designed to be able to run both HASH and Hasha ASCON algorithms while maintaining a compact area budget and competitive performance. It supports multiple message size and can be tuned to meet different power and energy requirements based on the IoT specifications. The proposed processor has been evaluated in the context of an embedded system by using it as a co-processor for the soft-core processor Micro-Blaze. Evaluation results show that this design is able to run at a maximum of 200 MHz while consuming around 0.24W of power and taking 1083 LUT (Look-Up-Table) on Zynq located on the PYNQ-Z1 board. It outperforms the state-of-the-art implementation of the hash algorithm [13] that didn't have the reconfigurability, where ~2188 LUT were taken on ARTIX-7 chip while running at 158.9 MHz. The proposed processor is designed with Chisel, which is a fully open-source hardware construction language. It is also implemented with both commercial and open-source ASIC design flows in various technology nodes. The evaluation results in ASIC shows that it can be used as a competitive candidate to be embedded as a security primitive IP as part of a SoC due to its compactness, flexibility and low power consumption.

3. ASCON Hash Function

The ASCON series is based on sponge duplex construction, providing both hashing and authenticated encryption with associated data functionality for systems. As shown in Fig. 1, the sponge construction is a mode of operation which builds a function mapping input with variable length to variable length output using a fixed-length permutation and on a padding rule. Sponge construction usually operates on a finite state by iterative applying the inner permutation to it, which is a generalization of both stream ciphers, which have a fixed input length, and hash functions, which have a fixed output length. The duplex construction is considered a sister of the sponge construction as it allows the alternation of input and output blocks at the same rate as the sponge construction [11].

The ASCON hash function [11] performs a permutation on any sized message to produce a digest or a fixed-length output of that function. ASCON hash is meant to be a low-power function with low resource usage to allow embedded devices to be secure. Lightweight cryptography functions focus on securing low-power devices such as IoT and microcontrollers. The ASCON function is performed using XOR, Rotation, and a set of constants, in addition to AND and NOT functions to optimize the substitution layer. The lightweight nature of the algorithm allows for a small hardware area, meaning the device is area efficient. Thus, multiple processors can be placed into a system with minimal overhead.

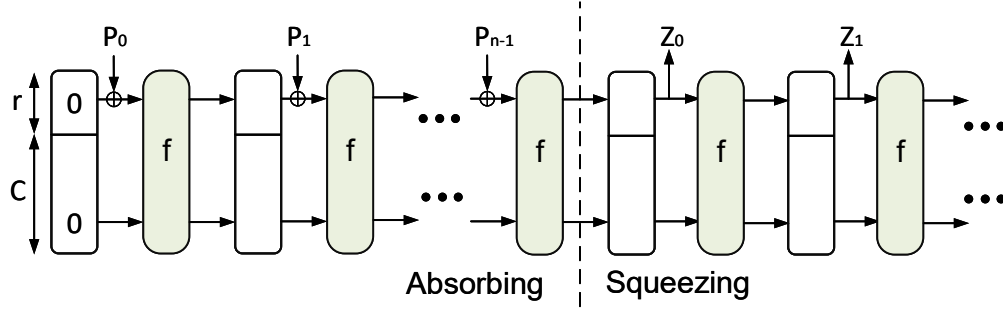


Figure 1: Sponge Construction in ASCON [11]

As it depends on sponge construction, ASCON hash algorithms consist of the initial stage, which absorbs the input message in a block fashion and performs ARX operations. The final stage squeezes the internal states to produce the last digest, the official hash function output. Finally, the ARX operations are sorted under permutation functions with differing rounds. The main difference between Hash and Hasha algorithms under ASCON submission is the total number of rounds per permutation function [11].

3.1. Initialization

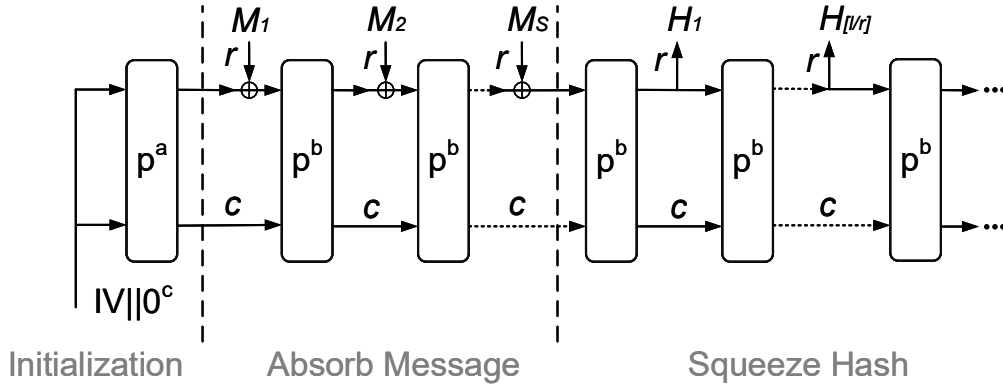


Figure 2: ASCON Hashing [11]

As shown in Figure. 2, both hash and hasha modes consist of three stages Initialization, Absorbing, and Squeezing. For the initialization stage, the permutation stage p^a receives 320-bit representing the initial state defined by a constant IV. The output of the initialization stage will be used to start the process of absorbing, which depends on segmenting the message into blocks and running each part at a time in the p^a function; the output of a block of the message will be an input state for the next block of the message until the permutation function absorbs the whole message. Once the last part of the message is processed, the squeezing process will start by running the permutation unit and producing a part of the output digest as shown in Figure. 2. The output digest size is 256-bit as both ASCON hash and ASCON hasha are meant for lightweight cryptographic-based applications. The size of the message block processed at a time is called the ratio, which is 64-bit. The power of the permutation stage represents the number of rounds this function will be processing the input of it. For instance, ASCON hasha number of rounds is 8 while ASCON hash is 12 rounds. The idea of proposing a processor that can run both modes is to create a processor that can offer both hash versions, which presents different security margins. The scenario of using such a processor depends on how much the operating battery can last and how many security challenges are around the system that uses the processor.

4. Architecture of RECO-ASCON

The flexibility offered by RECO-ASCON enables any system hosting this architecture to seamlessly switch between Hash and Hasha modes of ASCON without impacting the overall power consumption and performance of the system. To accomplish our objective, we utilized Chisel [48, 49], an high-level HDL based on Scala, for designing the circuit of the proposed processor, aligning with the academic trend towards open-source tools and reducing development time. Unlike Verilog, Chisel is a higher-level hardware construction language with advanced design abstraction and code generation capabilities, while Verilog is a lower-level hardware description language focused on describing circuits at the gate and register-transfer level. Chisel is a domain-specific language for digital circuit designs and is implemented as a class library of Scala general-purpose language [50]. It allows us to describe synthesizable digital circuits and simulate them seamlessly. Thanks to the power of Scala’s modern programming language paradigms, such as functional and object-oriented programming, Chisel offers higher expressivity that dramatically improves the productivity of the circuit design process. Many real-world taped-out designs, which include various RISC-V processors [51, 52] and Google’s edge TPU [53], and various research hardware designs [54, 55, 56, 57] are implemented in Chisel. One of the important aspects of Chisel is that it encourages test-driven development and offers fully integrated testing harnesses so that users can write unit tests in Scala. Chisel simulates the behavior of generated circuits using either an internal Scala-based simulator or an external simulator such as Verilator [58] that translates Verilog codes into C++ codes for faster simulation. Several studies confirmed that Chisel significantly reduces the code size, improves the code reusability, and incurs no performance penalty (the most cases, the same frequency) as Verilog [59, 60, 61, 62, 61].

As shown in Figure. 3, RECO-ASCON is a reconfigurable hashing processor that can be easily switched from the Hash mode to the Hasha mode of the ASCON processors, allowing a message to be hashed in either mode. The mode switching happens within one cycle. Moreover, the controller inside the RECO-ASCON is designed to prevent switching between modes while the data is processed. If a certain mode starts, the switching could happen once the current mode is finished digesting the input message.

Each component in the proposed processor is built from the ground up to create this processor, starting with small modules to be composed into larger ones. Thus, each of the three permutations was created [11] and then sequenced to create a permutation hardware block. This processor utilizes precomputed states and a state machine to perform the desired functions, modifying the permutation when necessary for either mode. Additionally, one permutation block is used to perform any permutation round necessary to minimize hardware needs. The processor consists of A FIFO to store the message from the softcore processor, which will input the constant permutation stage to perform the initialization stage. The output of the initialization stage will be processed by both substitution permutation and linear permutation stages, which are part of the permutation unit p^a . As discussed in section 3 number of rounds depends on which ASCON hash is running. As a final stage, the output digest is processed by output registers, and the Mux unit selects which part of the message will be handed back to the softcore processor for further processing in the system.

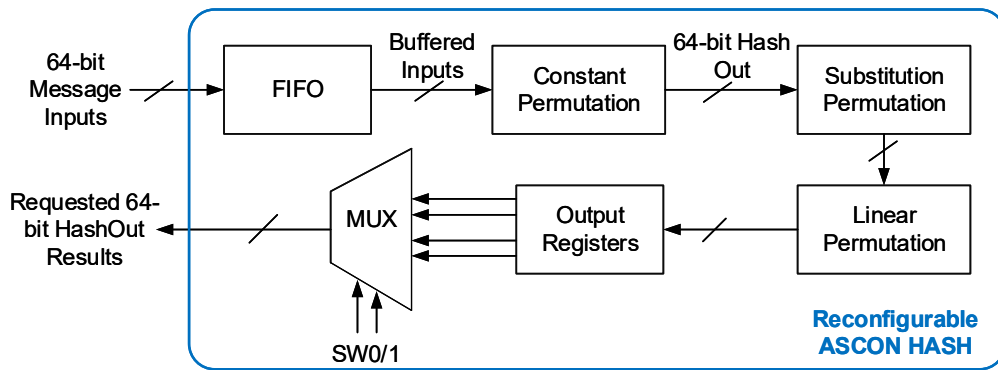


Figure 3: The Architecture of RECO-ASCON Design

In this paper, we study the implementation of ASCON hash functions hash and hasha on both softcore processor-based solutions vs. low-power embedded systems. We used the proposed processor RECO-ASCON as a co-processor for the softcore processor MicroBlaze as shown in Fig. 4, MicroBlaze microprocessor is surrounded by local memory and debug portal while an AXI crossbar allows several coprocessors of RECO-ASCON to be connected to MicroB-

laze. Moreover, the system has a control units which take care of resetting these coprocessors. In addition, there is a PLL clock generating unit [63] that boosts the maximum frequency as PYNQ-Z1 crystal produces 100 MHz while the coprocessors cluster runs in the 200 MHz to 250 MHz. MicroBlaze stream data to the RECO-ASCON cluster through the AXI-Crossbar while the cluster sends back the output digest to MicroBlaze using a handshake approach. There are a set of status registers defined at the driver level for the RECO-ASCON cluster. These monitor the data processing inside the cluster and guarantee data correctness without any possibility of generating hazards. For measuring the total number of cycles, an AXI-timer module has been added, which is used to check the total number of cycles the RECO-ASCON takes to produce the output digest based on the size of the message sent from MicroBlaze to the cluster. The reason for using AXI-timer is the current microprocessor doesn't support operating systems such as Linux.

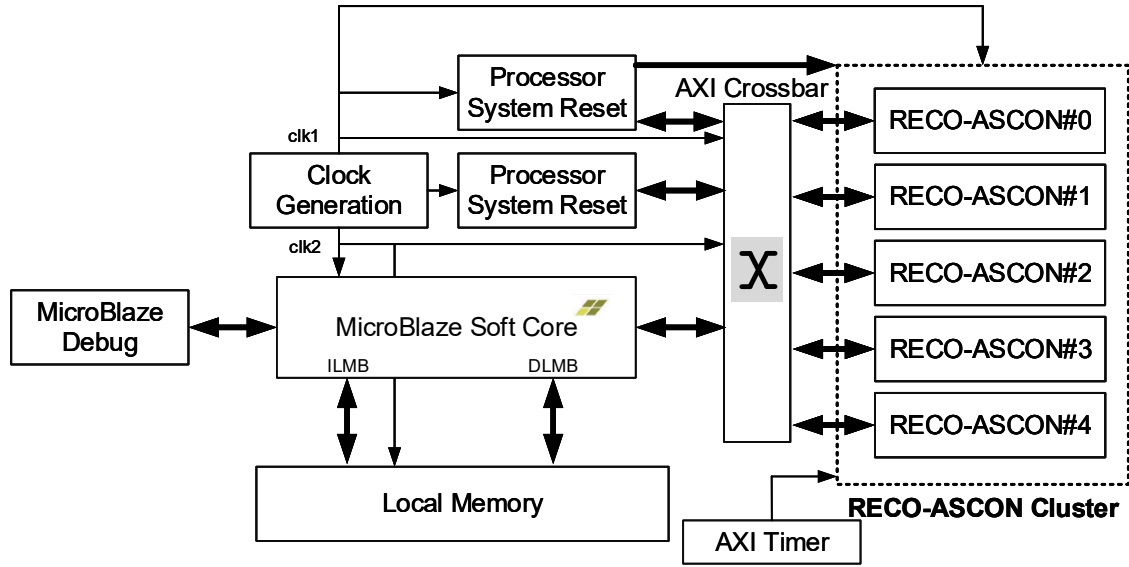


Figure 4: System view of integrating RECO-ASCON cluster with MicroBlaze Soft Core on PYNQ FPGA.

This implementation with a rate being 64 and a capacity of 256 with a design such that signals are sent to indicate that a section of the hash is complete. The 64-bit messages are sent to a buffer FIFO, so the processor does not take messages when computing a hash. Output is delayed until a control signal is sent to indicate the final message to be sent entering into the queue. Once the FIFO is empty. The processor begins the digest step. Finally, a done signal is sent back to the software system to indicate that it can receive the sections of the completed hash from the output register that stored them as a 256 hash value. This is a light overview of how the RECO-ASCON cluster is fed data from the MicroBlaze microprocessor through AXI-bus.

5. Evaluation Results

To test the hardware and software implementations from the PYNQ and Raspberry Pi 3, modules and timer functions are incorporated to collect the cycles per 64-bit padded block. Comparing these results shows the efficiency of offloading the hashing function from a heavy software process to a lightweight hardware solution.

The current formula for the number of cycles taken is the following. Where N = number of cycles, M = number of 64-bit messages, D = number of cycles to Digest Output, S = number of cycles for Setup.

$$N = 25M + D + S$$

The current optimization of the processor's performance has each message input scaling linearly per 64-bit padded input. Stating a null input padded and a full 64-bit padded message will take the same time because they are considered the same message input. Thus each additional increment of 64 bits will result in only 25 cycles.

Digest and setup cycles (D, S) are 75 and 3, respectively. Thus, the output is divided into four sections, and the processor takes three additional rounds to complete the hash. Calculating for a default 100 MHz clock to find the throughput results just under 256 Mbits/s. This assumes there is no system delay due to the FIFO signaling it is full. This is seen as such in the simulated results. A null padded message input alone takes 103 cycles. The system functions from a single 200 MHz clock shared between the Micro-blaze soft processor and the RECO-ASCON processor.

5.1. Raspberry Pi 3

The Raspberry Pi 3 Model B [64] has a quad-core processor, and each core can support up to four threads. This means that the Raspberry Pi 3 Model B can handle a total of 16 threads concurrently, with each core executing up to four threads using simultaneous multithreading (SMT) technology [65]. It can run the ASCON hash function algorithms on a single core per operation. The process can be duplicated by modifying the test code to include the time run total and forking, and multiple copies of the same code will be requested to run on several threads. Using these together, the Raspberry Pi can be benchmarked again. Similar values persist during this test for larger messages, where smaller messages can be significantly affected, leading to an increase of cycles up to 7x the original cycle count. However, the total time it takes to complete this task increases after more than 4 threads are present. As this Pi has 4 cores, these tasks require OS scheduling and context switching between threads to run all required threads.

Table 1: Multiple threads running the ASCON processor's test

Number of Threads	Total Time (s)	Efficiency (1/s)	Cores Used
1	3.547476	0.281890	1
2	3.614112	0.553386	2
4	3.731632	1.071917	4
8	7.58507	1.054704	4
16	14.419913	1.109577	4

Table 1 shows the number of threads used for a run, the total time it took to complete the task, the efficiency for the program being threads per second, and core used being derived from the efficiency value divided by the efficiency of one thread. Comparing 4 threads to 8 threads and 8 threads to 16 threads, the time to complete the task doubles, meaning all cores are running at full potential. Comparing 2 and 4 threads shows almost equivalent times, as only a fork operation and safe closing logic is added, leading to a small delay and the number of threads does not exceed the number of cores.

The total resources used for mapping the proposed RECO-ASCON processor on the PYNQ kit is 339 slices out of 13300 on-chip, which is 1145 Slice LUT for the combinational part of the processor out of a total of 53200 Slice LUT on-chip. Therefore, the used Slice LUT was divided as 1101 LUT as logic, while 44 acted as a memory for storing the intermediate coefficients. This result shows that the proposed processor utilizes less than 3% of the PYNQ's FPGA resources. Thus, in theory, around 40 processors can be packed into a single PYNQ. Due to the software overhang, the PYNQ system does not run at the expected throughput.

The current process sends hard-coded array data to represent the input message within the Xilinx SDK and read/write to the control registers of the AXI bus. With as much consideration to optimizing the software control while keeping the integrity of the test. The Table 2 shows that while the number of cycles for a null input is about four times as much as it takes the processor to compute via simulation. The PYNQ still outperforms the Raspberry Pi in cycles per byte by roughly six times. Since the rate is 64 bits, one block of the message sent to the processor should be that size, which is why padding is beneficial. Padding allows the message to fit perfectly in the rate, even if it does not match in size. In addition, these padded blocks are useful to measure a difference in cycles since an additional absorption permutation is needed to absorb the full message.

There is an outstanding difference in performance per block iteration in cycle increase. The cycle per block is consistent on the PYNQ because the processor takes a significant number of cycles less than the software functions controlling them. Reading and writing specified control bits from the software take a significant amount of time compared to the calculation. This is a vast improvement from a typical software solution since the hashing function will not be the constraining factor in data processing from the CPU. When using a USB power monitor connected to the PYNQ, there is no noticeable change in idle power from the USB port. Therefore, the estimated power by the

Table 2: Hash/Hasha Modes

Tests	Rpi3 Hash	Rpi3 Hasha	PYNQ Hash/Hasha
Cycles (1 block)	1833	1480	445
Cycles per byte (1 block)	229	185	55.6
Cycles (2 block)	2268	1800	646
Cycles per byte (2 block)	142	113	14.4
Cycle Difference	435	320	201

¹ block = 64 bit padded message

Vivado tool is the same as the calculated power by the USB power monitor, which is 0.242 W. Around 56% of them is dynamic power. At the same time, the rest is static, which is 44% of the total power. The dynamic power is distributed as 11% for clocks, 14% for signals, 8% for logic, 67% for using BRAM. The consumed power by BRAM is mainly for the Micro-Blaze soft-core processor and its peripherals as we the proposed RECO-ASCON processor is mainly consumed LUTs for both logic and memory activities on-chip.

Being near indistinguishable from typical fluctuations in the USB power out. With better tools at our disposal in the future, it will be easier to measure the output power usage. In this paper, we make use of both the PYNQ-Z2 board to compare the results with the Raspberry Pi 3. To increase overall throughput and performance computation. We are able to use multiple of the processors for the same implementation. While staying at 200 MHz, the limit currently for the PYNQ board is 5 instances of the processor as shown in Fig. 4. This impacts area by an overall increase of 4 times but throughput increase by a multiple of 5. The total area is a multiple of 4 because micro-blaze needs to manage the addresses of the processors through the crossbar IP and that adds more in resources. If it did not then the scale ability would strictly be linear. The system will be able to micro manage the 5 new processors and process data independently of each other if the system has multiple requests. There is similar results when porting the block diagram over to a NEXYS 4 DDR board. Both support 5 processors at 200 MHz with similar resource allocation. Meaning the system can be reasonably ported to other board.

5.2. ASIC Evaluation Results

The ultimate goal of IoT chips that sit at the edge is to enable low-power and high-security computations with low costs. To evaluate that the proposed ASCON hash processor is able to be embedded as a candidate IP for securing IoT system-on-chip (SoC), we evaluate the design with the standard ASIC automatic place and route (APR) flows with various technology nodes. It has been shown that 28nm to be still a long-lived node for semiconductor applications in the next five years, we thus first implement the design with 28nm industry technology node. The synthesis is run with Cadence Genus©, and physical design is carried out by Cadence Innovus©. Static timing analysis (STA) is conducted in Cadence Tempus©. To ensure an end-to-end approach delivery of the proposed design in a fully open-sourced fashion, we also evaluated the design with OpenLane [66], an automated RTL to GDSII flow based on all open-source EDA tools, with 45nm NanGate open cell library [67] built upon the open-source technology node FreePDK45 [68].

The gate-level netlist from synthesis tools is fed to the physical implementation tools to run the floorplanning and P&R (place and route) steps. Extraction is conducted for running STA. The area information together with gate count is taken from the P&R result. Timing is checked based on sign-off requirements, where worse-case scenarios are considered. The design has been implemented under worse design corners such as SS/FF and high/low temperature combinations that are available from the corresponding technology libraries. The maximum clock frequency is sampled to ensure that the timing passes for both setup and hold checks. The power is taken as an average value with the power analysis tools from both tool suites. It is based on vectorless analysis. **We opt to use gate count instead of actual area for comparisons for the following reasons. Firstly, gate count holds significance for cryptography designers as it directly influences the size, complexity, and efficiency of hardware implementations. A lower gate count often correlates with reduced power consumption, improved performance, and potentially lower production costs, thereby**

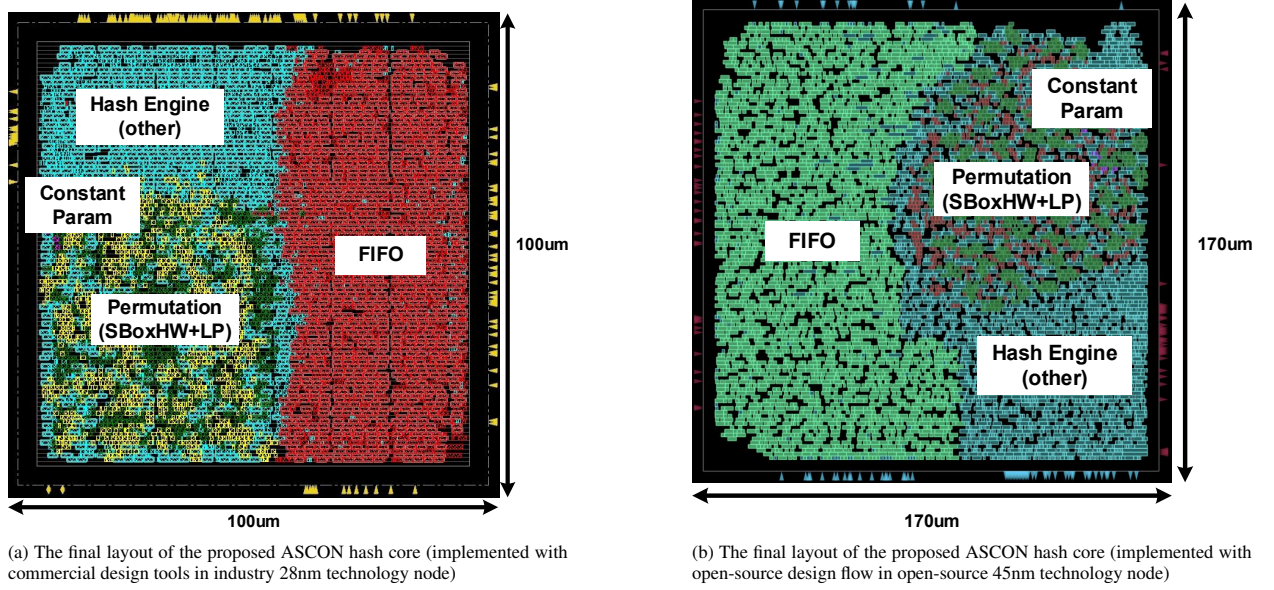


Figure 5: The layout of the final hash core design implemented with 28nm and 45nm technology nodes with commercial and open-source design flows. The core utilization has been kept as ~60%

playing a crucial role in optimizing cryptographic systems. Secondly, when comparing implementations across different technology nodes, it becomes challenging and unfair to directly compare actual chip areas. Therefore, gate count serves as a metric reflecting design complexity and as a proxy for making valid comparisons. Lastly, gate count is widely used and accepted within the cryptography architecture community. Given that the work we are comparing against also employs gate count, utilizing this metric facilitates fair comparisons with their findings.

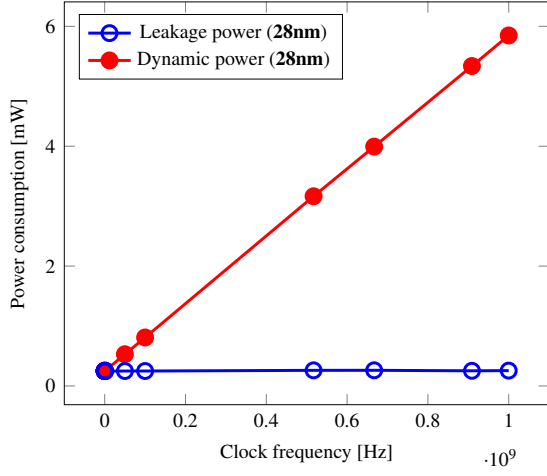
Fig. 5 shows the layouts of the proposed design in 28nm and 45nm technology nodes respectively. Submodules of the design has been highlighted with different colors. As can be seen, the majority of the area has been taken by FIFO, and the has engine takes about half of the area. But the design still achieves good compactness by taking only $9170.82\mu m^2$ in 28nm and $18225\mu m^2$ in 45nm. The maximum clock frequency in both designs are over 1GHz. Due to low power requirements, certain IoT domains such as sensing nodes run at low frequencies. Fig. 6 gives the power breakdown at various frequencies ranging from few Hz to GHz. The corresponding power consumption is within Milliwatts range at high frequencies (over 1GHz) and few hundreds Microwatts. It is worth to mention that the leakage power (denoted as blue line) can be further reduced by introducing multi-Vt and power-gating design techniques.

The throughput can be calculated with the latency and the maximum frequency and the energy per bit is acquired by dividing the operation power with the throughput of each instance (1), (2). The throughput and energy consumed to process 8192 bits (8192/64 blocks) are listed in Table 3. Under 28nm technology node, the maximum frequency is 1.12GHz, which leads to that the throughput is 2.83Gb/s and the energy efficiency is as high as 2.2 pJ/bit. For the 45nm technology node, the maximum frequency is 1 GHz, which leads to that the throughput is 2.53 Gb/s and the energy efficiency is 10.4 pJ/bit. On the other hand, the throughput on the FPGA is 0.5 Gb/s at 200MHz while consuming 0.48 nJ/bit of the energy.

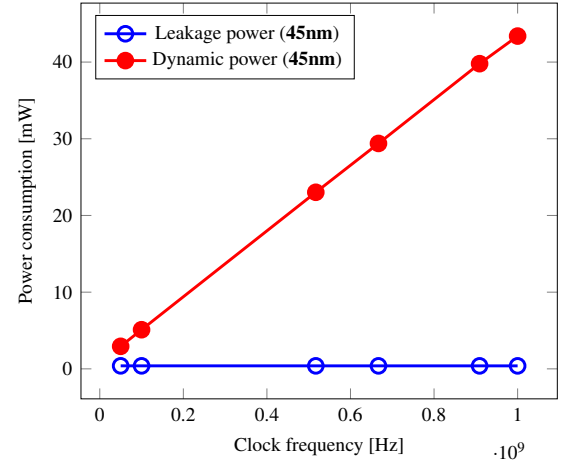
$$Throughput = \frac{MaxFreq \cdot DigestBits}{L} \quad (1)$$

$$Energy/Bit = \frac{Power}{Throughput} \quad (2)$$

While most LWC designs offer one functionality only, the proposed RECO-ASCON offers two Hashing instances, Hash and Hasha. It is twice the number of the second flexible PRO-GAGE [30]. Supporting multiple instances offers



(a) Power breakdown at different frequencies for 28nm technology node (commercial EDA tools).



(b) Power breakdown at different frequencies for 45nm technology node (Open-source flow).

Figure 6: Power breakdown of the proposed design in two technology nodes.

flexible options for IoT devices and better compatibility. RECO-ASCON also features the highest throughput which is critical for establishing gigabit wireless connection between IoT devices. Although some designs have smaller power consumption, their energy efficiency are much poorer. RECO-ASCON achieves the highest energy efficiency (smallest energy per bit in 28nm). While PRO-GAGE [30] achieves the highest energy efficiency among the state of the art implementations in Table 3, the proposed processor is 38% more energy efficient while taking 13% less area by running at a higher clock frequency of 1.12GHz (19% higher). In addition, according to the message size and clock frequency, the proposed process can be tuned and tailored to satisfy different energy requirements as shown in Fig. 7. In summary, RECO-ASCON achieves the great energy efficiency by supporting dual hash modes, and it takes only 5.7K gates (in 28nm) or 8.76K gates (in 45nm) while running at a faster frequency. The compactness of the design makes it suitable to be deployed as part of a IoT system that supports a wide range of applications.

Table 3: Comparison against other LWC hash designs, the reported energy is based on 8192 bits digestion.

LWC design	FPGA				ASIC				
	Chip	Max.Freq (MHz)	LUT	FF	Technology	Area (Gate Count)	Max.Freq (MHz)	Power (μ W)	Energy Efficiency (nJ/bit)
RECO-ASCON (Proposed)	ZYNQ XC7Z020	200	1083	681	28 nm (industry)	5690	1120	6210	0.0022
PRO-GAGE [30]	Artix-7	250	436	514	45 nm (FreePDK)	8755	1000	26400	0.0104
GAGE [69]	Artix-7	250	226	120	32/28 nm (EDK)	6553	909	655	0.0036
ACE [70]	Spartan 3	181	381	327	65 nm (industry)	~4600	705	–	20.1
WAGE [71]	–	–	–	–	65 nm (industry)	~3290	1120	–	13.0
Subterranean 2.0 [72]	–	–	–	–	STM 40 nm (industry)	~4165.5	100	432.4	–

6. Conclusion

Overall, we have successfully designed and implemented a lightweight cryptographic processor RECO-ASCON, capable of switching between the ASCON hashing algorithms for each set of input messages. Furthermore, we incorporated the Micro-Blaze soft-core processor as a test system to feed target data into the proposed multi-core RECO-ASCON processor cluster and retrieve the appropriate hash values for functional verification. Produced results for data efficiency in cycles per 64-bit padded message block compared to a software implementation on a Raspberry Pi

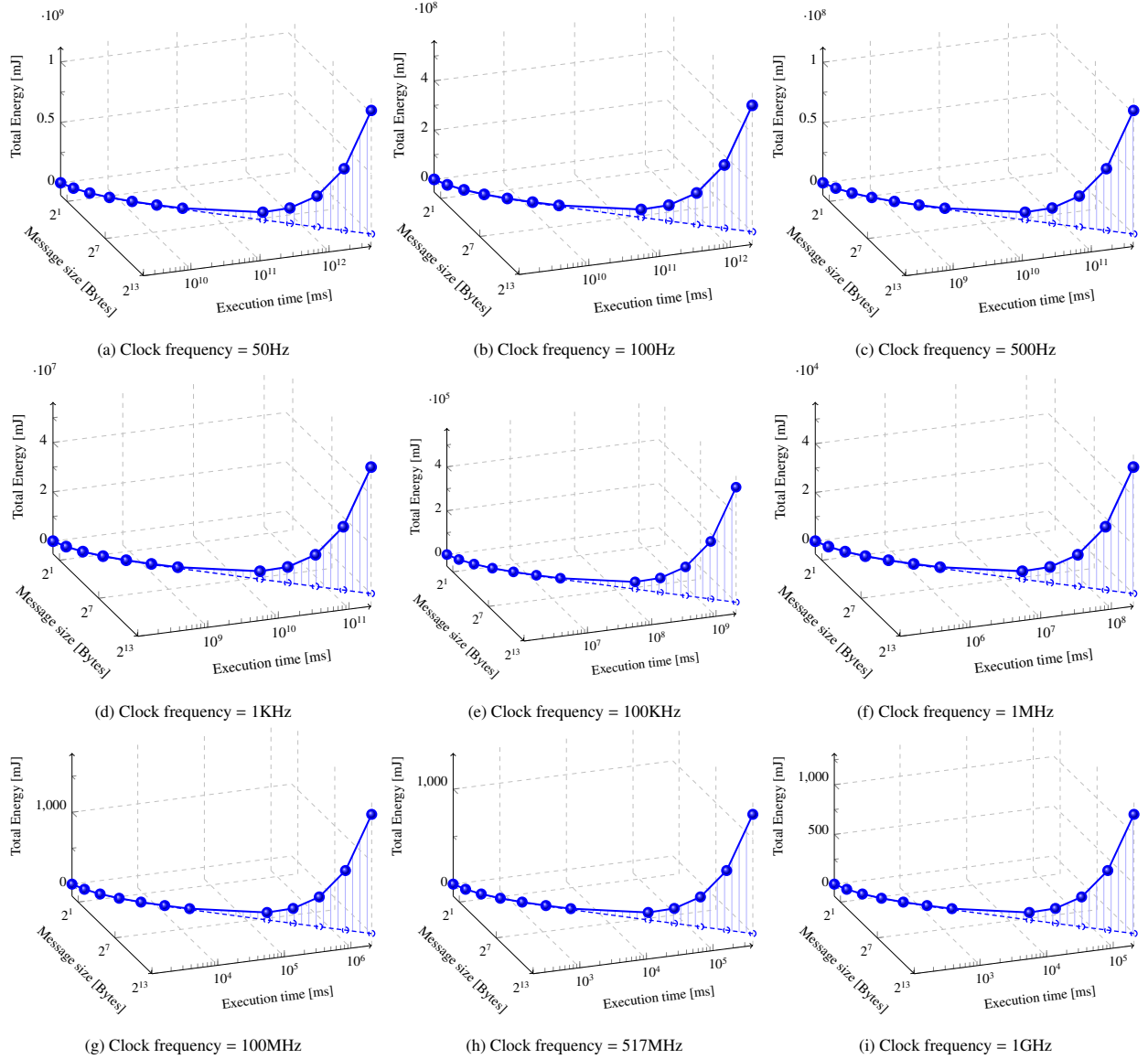


Figure 7: Performance and energy comparison at various frequencies based on 28nm implementations. The proposed processor can support different energy levels by tuning the message size and clock frequency. This adds another dimension of flexibility to embrace the diversity of the IoT system requirements.

3 shows that using the reconfigurable RECO-ASCON on FPGA can be a promising solution for securing constrained IoT devices. Moreover, in this paper, we proved that the proposed RECO-ASCON processor on the Nexsys4-DDR Xilinx Kit, which uses the ARTIX-7 chip, would cost $\approx$ two times fewer resources and run $1.25 \times$ faster than the existing implementation of just one of the ASCON hash functions on the same family of the chip while consuming 240 mW at maximum. We evaluate the proposed RECO-ASCON processor in two different technology nodes with both commercial and open-source EDA tools. In 28nm industry node, the proposed design is implemented with Cadence design suites and is able to run at a maximum of 1.12GHz while taking only 5.7K gates. In 45nm open-source technology, it is implemented with the open-source design tools and still achieves 1GHz maximum frequency. Overall, the proposed processor is compact, reconfigurable, fast, and energy efficient. This makes it a great candidate for securing various IoT systems.

Acknowledgement

This work was partially supported by the U.S. Department of Energy, Office of Science, under contract DE-AC02-06CH11357. In addition, this work was partially supported by National Science Foundation of China under Grant No. 62201340, in part by a CCF-Tencent Open Fund No. RAGR20220114, and in part by a startup funding from Shanghai Jiao Tong University. Authors would like to thank the anonymous reviewers for their valuable feedback.

References

- [1] M. Capra, R. Peloso, G. Masera, M. Ruoch, M. Martina, Edge computing: A survey on the hardware requirements in the internet of things world, *Future Internet* 11 (4) (2019) 100.
- [2] W. Shi, S. Dustdar, The promise of edge computing, *Computer* 49 (5) (2016) 78–81.
- [3] L. P. Oliveira, L. N. Carneiro, et al., Security, power consumption and simulations in iot device networks: A systematic review, in: *International Conference on Advanced Information Networking and Applications*, Springer, 2022, pp. 370–379.
- [4] X. Guo, V. Verma, P. Gonzalez-Guerrero, M. R. Stan, When “things” get older: exploring circuit aging in iot applications, in: *2018 19th International Symposium on Quality Electronic Design (ISQED)*, IEEE, 2018, pp. 296–301.
- [5] V. Da Rocha, N. Moura, J. Gava, V. Bandeira, L. Ost, R. Reis, R. Garibotti, Soft error reliability assessment of lightweight cryptographic algorithms for iot edge devices, in: *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, IEEE, 2022, pp. 457–460.
- [6] A. E. Omolara, A. Alabdulatif, O. I. Abiodun, M. Alawida, A. Alabdulatif, H. Arshad, et al., The internet of things security: A survey encompassing unexplored areas and new insights, *Computers & Security* 112 (2022) 102494.
- [7] Y. Xiao, Y. Jia, C. Liu, X. Cheng, J. Yu, W. Lv, Edge computing security: State of the art and challenges, *Proceedings of the IEEE* 107 (8) (2019) 1608–1631.
- [8] S. Rani, D. Gupta, S. Garg, M. Jalilpiran, M. S. Hossain, Consumer electronic devices: Evolution and edge security solutions, *IEEE Consumer Electronics Magazine*.
- [9] K.-A. Shim, A survey of public-key cryptographic primitives in wireless sensor networks, *IEEE Communications Surveys & Tutorials* 18 (1) (2016) 577–601. doi:10.1109/COMST.2015.2459691.
- [10] NIST, Submission requirements and evaluation criteria for the lightweight cryptography standardization process, accessed: 2018-08.
- [11] C. Dobraunig, M. Eichlseder, F. Mendel, M. Schl  ffer, *Ascon v1.2* (2019 [Online]).
URL <https://ascon.iaik.tugraz.at/publications.html>
- [12] P. Joshi, B. Mazumdar, Single event transient fault analysis of ELEPHANT cipher, CoRR abs/2106.09536. arXiv:2106.09536.
URL <https://arxiv.org/abs/2106.09536>
- [13] B. Rezvani, F. Coleman, S. Sachin, W. Diehl, Hardware implementations of nist lightweight cryptographic candidates: A first look, *Cryptology ePrint Archive*, Report 2019/824 (2019).
URL <https://eprint.iacr.org/2019/824>
- [14] M. Hell, T. Johansson, A. Maximov, W. Meier, J. Sonnerup, H. Yoshida, Grain-128aeadv2- a lightweight aead stream cipher (2019).
- [15] B. Udvarhelyi, O. Bronchain, F.-X. Standaert, Security analysis of deterministic re-keying with masking and shuffling: Application to isap, in: S. Bhasin, F. De Santis (Eds.), *Constructive Side-Channel Analysis and Secure Design*, Springer International Publishing, Cham, 2021, pp. 168–183.
- [16] Z. Bao, A. Chakraborti, N. Datta, J. Guo, M. Nandi, T. Peyrin, K. Yasuda, Photo-bettle: Authenticated encryption and hash family (2019).
- [17] A. Deprez, E. Andreeva, J. M. B. Mera, A. Karmakar, A. Purnal, Optimized software implementations for the lightweight encryption scheme forkae, *Cryptology ePrint Archive*, Report 2020/1295, <https://ia.cr/2020/1295> (2020).
- [18] C. Berierle, A. Biryukov, L. C. dos Santos, J. GroBschadl, L. Perrin, A. Udovenko, V. Velichkov, Q. Wang, Schwaemm and esch: Lightweight authenticated encryption and hashing using the sparkle permutation family (2019).
- [19] H. Wu, T. Huang, Tinyjambu: A family of lightweight authenticated encryption algorithms (version 2) (2021).
- [20] J. Daemen, S. Hoffert, M. Peeters, G. V. Assche, R. V. Keer, Xoodyak, a lightweight cryptographic scheme (2020).
- [21] H. Madushan, I. Salam, J. Alawatuogoda, A review of the nist lightweight cryptography finalists and their fault analyses, *Electronics* 11 (24) (2022) 4199.
- [22] M.-M. Bidmeshki, G. R. Reddy, L. Zhou, J. Rajendran, Y. Makris, Hardware-based attacks to compromise the cryptographic security of an election system, in: *2016 IEEE 34th International Conference on Computer Design (ICCD)*, 2016, pp. 153–156. doi:10.1109/ICCD.2016.7753274.
- [23] D. O. Danilo Gligoroski, Hristina Mihajloska, *Gage and ingage v1.0*.
URL <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/GAGEandInGAGE-spec.pdf>
- [24] W. J. Buchanan, L. Maglaras, Review of the nist light-weight cryptography finalist, arXiv preprint arXiv:2303.14785.
- [25] Athena, *Caesar hardware api implementation*, accessed: 2021-02-28.
URL https://cryptography.gmu.edu/athena/index.php?id=CAESAR_source_codes
- [26] A. Sathvik, T. Rahul, A. Bakshi, V. Pudi, Hardware implementation of spoc-128, *Cryptology ePrint Archive*, Report 2022/119, <https://ia.cr/2022/119> (2022).
- [27] J. Anderson, Y. Alkabani, T. El-Ghazawi, Recipe: A pe for reconfigurable lightweight cryptography, in: *2021 IEEE 34th International System-on-Chip Conference (SOCC)*, IEEE, 2021, pp. 176–181.
- [28] S. Banik, S. K. Pandey, T. Peyrin, Y. Sasaki, S. M. Sim, Y. Todo, Gift: A small present, *Cryptology ePrint Archive*, Report 2017/622, <https://ia.cr/2017/622> (2017).

- [29] J. Sönnerup, M. Hell, M. Sönnerup, R. Khattar, Efficient hardware implementations of grain-128aead, in: F. Hao, S. Ruj, S. Sen Gupta (Eds.), *Progress in Cryptology – INDOCRYPT 2019*, Springer International Publishing, Cham, 2019, pp. 495–513.
- [30] M. El-Hadedy, M. Margala, S. Mosanu, D. Gligorosk, W.-M. Hwu, Pro-gage: A high performance compact gage hash function processor for small space technology, in: 2021 IEEE Space Computing Conference (SCC), 2021, pp. 9–16. doi:10.1109/SCC49971.2021.00009.
- [31] M. El-Hadedy, X. Guo, Realse: Reconfigurable lightweight security engines for trusted edge devices, in: 2022 IEEE 4th International Conference on Circuits and Systems (ICCS), IEEE, 2022, pp. 7–12.
- [32] A. Alfrhan, T. Moulahi, A. Alabdulatif, Comparative study on hash functions for lightweight blockchain in internet of things (iot), *Blockchain: Research and Applications* 2 (4) (2021) 100036.
- [33] C. Biswas, U. D. Gupta, M. M. Haque, An efficient algorithm for confidentiality, integrity and authentication using hybrid cryptography and steganography, in: 2019 International Conference on Electrical, Computer and Communication Engineering (ECCE), 2019, pp. 1–5. doi:10.1109/ECACE.2019.8679136.
- [34] B. Hu, Q. Xie, Y. Li, Automatic verification of password-based authentication protocols using smart card, in: 2011 International Conference of Information Technology, Computer Engineering and Management Sciences, Vol. 1, 2011, pp. 34–39. doi:10.1109/ICM.2011.286.
- [35] A. Mishra, K. Kishore, V. Kumar, An eclipse plugin to assist learners in selecting hash functions, in: 2016 IEEE Eighth International Conference on Technology for Education (T4E), 2016, pp. 172–175. doi:10.1109/T4E.2016.043.
- [36] A. L. Zobrist, Ordered minimal perfect hash of the human genome and implications for duplicate finding, in: 2009 WRI World Congress on Computer Science and Information Engineering, Vol. 4, 2009, pp. 106–111. doi:10.1109/CSIE.2009.1070.
- [37] V. Skala, M. Kucha, The hash function and the principle of duality, in: *Proceedings. Computer Graphics International* 2001, 2001, pp. 167–174. doi:10.1109/CGI.2001.934671.
- [38] D. Wang, Y. Jiang, H. Song, F. He, M. Gu, J. Sun, Verification of implementations of cryptographic hash functions, *IEEE Access* 5 (2017) 7816–7825. doi:10.1109/ACCESS.2017.2697918.
- [39] I. G. A. K. Gemeliarana, R. F. Sari, Evaluation of proof of work (pow) blockchains security network on selfish mining, in: 2018 International Seminar on Research of Information Technology and Intelligent Systems (ISRITI), 2018, pp. 126–130. doi:10.1109/ISRITI.2018.8864381.
- [40] Xilinx, *PYNQ-Z1 Setup Guide*, accessed: 2018. URL "https://pynq.readthedocs.io/en/v2.2.1/getting_started/pynq_z1_setup.html"
- [41] X. Guo, M. El-Hadedy, S. Mosanu, X. Wei, K. Skadron, M. R. Stan, Agile-aes: Implementation of configurable aes primitive with agile design approach, *Integration* 85 (2022) 87–96.
- [42] T. Polk, S. Turner, P. E. Hoffman, *Security Considerations for the SHA-0 and SHA-1 Message-Digest Algorithms*, RFC 6194 (Mar. 2011). doi:10.17487/RFC6194. URL https://www.rfc-editor.org/info/rfc6194
- [43] C. Xiao-hui, D. Jian-zhi, Design of sha-1 algorithm based on fpga, in: 2010 Second International Conference on Networks Security, Wireless Communications and Trusted Computing, Vol. 1, 2010, pp. 532–534. doi:10.1109/NSWCTC.2010.131.
- [44] W. Sun, H. Guo, H. He, Z. Dai, Design and optimized implementation of the sha-2(256, 384, 512) hash algorithms, in: 2007 7th International Conference on ASIC, 2007, pp. 858–861. doi:10.1109/ICASIC.2007.4415766.
- [45] X. Wu, S. Li, High throughput design and implementation of sha-3 hash algorithm, in: 2017 International Conference on Electron Devices and Solid-State Circuits (EDSSC), 2017, pp. 1–2. doi:10.1109/EDSSC.2017.8126446.
- [46] M. A. Alahmad, I. F. Alshaikhli, M. Nandi, Multicollisions in sponge construction, in: 2013 International Conference on Informatics and Creative Multimedia, 2013, pp. 215–219. doi:10.1109/ICICM.2013.44.
- [47] X. Wei, M. El-Hadedy, S. Masanu, Z. Zhu, W.-M. Hwu, X. Guo, Reco-hcon: A high-throughput reconfigurable compact ascon processor for trusted iot, in: 2022 35th IEEE International System-on-Chip Conference (SOCC), IEEE, 2022, pp. 1–6.
- [48] J. Bachrach, H. Vo, B. Richards, Y. L. D. a. d. . Design, Chisel: constructing hardware in a scala embedded language, *DAC Design Automation Conference* (2012) 1212–1221.
- [49] J. J. Bachrach, K. Asanović, Chisel 3.0 tutorial, EECS Department, UC Berkeley, Tech. Rep.
- [50] M. Odersky, L. Spoon, B. Venners, *Programming in scala*, Artima Inc, 2008.
- [51] K. Asanović, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, J. Koenig, Y. Lee, E. Love, M. Maas, A. Magyar, H. Mao, M. Moreto, A. Ou, D. A. Patterson, B. Richards, C. Schmidt, S. Twigg, H. Vo, A. Waterman, *The Rocket chip generator*, Tech. Rep. UCB/EECS-2016-17, EECS Department, University of California, Berkeley (Apr 2016). URL http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html
- [52] C. Celio, P.-F. Chiu, B. Nikolic, D. A. Patterson, K. Asanovic, BOOMv2: an open-source out-of-order RISC-V core, in: *First Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2017.
- [53] Google, *Edge TPU: Google's purpose-built ASIC designed to run inference at the edge*. URL "https://cloud.google.com/edge-tpu"
- [54] L. Di Tucci, D. Conficconi, A. Comodi, S. Hofmeyr, D. Donofrio, M. D. Santambrogio, A parallel, energy efficient hardware architecture for the merAligner on FPGA using Chisel HCL, in: 2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), IEEE, 2018, pp. 214–217.
- [55] F. Serre, M. Püschel, A DSL-based FFT hardware generator in Scala, in: 2018 28th International Conference on Field Programmable Logic and Applications (FPL), IEEE, 2018, pp. 315–3157.
- [56] T. Nowatzki, V. Gangadhar, N. Ardalani, K. Sankaralingam, Stream-dataflow acceleration, in: 2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA), IEEE, 2017, pp. 416–429.
- [57] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, K. Olukotun, Plasticine: A reconfigurable architecture for parallel patterns, in: 2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA), IEEE, 2017, pp. 389–402.
- [58] W. Snyder, Verilator: Open simulation-growing up, DVClub Bristol.

- [59] X. Wei, X. Guo, Beyond verilog: Evaluating chisel versus high-level synthesis with tiny designs, in: 2022 23rd International Symposium on Quality Electronic Design (ISQED), IEEE, 2022, pp. 1–1.
- [60] O. Arcas-Abella, G. Ndu, N. Sonmez, M. Ghasempour, A. Armejach, J. Navaridas, W. Song, J. Mawer, A. Cristal, M. Luján, An empirical evaluation of high-level synthesis languages and tools for database acceleration, in: 2014 24th International Conference on Field Programmable Logic and Applications (FPL), IEEE, 2014, pp. 1–8.
- [61] S. Mosanu, X. Guo, M. El-Hadedy, L. Anghel, M. Stan, Flexi-AES: A highly-parameterizable cipher for a wide range of design constraints, in: 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), IEEE, 2019, pp. 338–338.
- [62] H. Kwon, T. Krishna, OpenSMART: Single-cycle multi-hop NoC generator in BSV and Chisel, in: 2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), IEEE, 2017, pp. 195–204.
- [63] S. Edway, R. K. Manjunath, Design and simulation of fpga based all digital phase locked loop (adpll), in: 2017 3rd International Conference on Applied and Theoretical Computing and Communication Technology (iCATccT), 2017, pp. 259–263. doi:10.1109/ICATCCT.2017.8389144.
- [64] Raspberry pi 3 model b, <https://www.raspberrypi.org/products/raspberry-pi-3-model-b/> (2016).
- [65] D. M. Tullsen, S. J. Eggers, H. M. Levy, R. E. Lo, Simultaneous multithreading: Maximizing on-chip parallelism, IEEE Micro 17 (2) (1996) 66–73. doi:10.1109/40.486465.
- [66] A. Ghazy, M. Shalan, Openlane: The open-source digital asic implementation flow, in: Proc. Workshop on Open-Source EDA Technol.(WOSET), 2020.
- [67] Nangate 45nm library, https://projects.si2.org/openeda.si2.org/help/group_ld.php?group=63%7D.
- [68] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, et al., Freepdk: An open-source variation-aware design kit, in: 2007 IEEE international conference on Microelectronic Systems Education (MSE'07), IEEE, 2007, pp. 173–174.
- [69] M. El-Hadedy, M. Margala, S. Mosanu, D. Gligoroski, J. Xiong, W. M. Hwu, Micro - gage: A low-power compact gage hash function processor for iot applications, in: 2020 27th IEEE International Conference on Electronics, Circuits and Systems (ICECS), 2020, pp. 1–4. doi:10.1109/ICECS49266.2020.9294948.
- [70] M. Aagaard, R. AlTawy, G. Gong, K. Mandal, R. Rohit, ACE: An Authenticated Encryption and Hash Algorithm, accessed: 2019-09-27 (September 27, 2019).
- [71] M. Aagaard, M. Sattarov, N. Zidaric, Hardware Design and Analysis of the ACE and WAGE Ciphers, accessed: 2019-01-29 (January 29, 2019).
- [72] J. Daemen, P. M. C. Massolino, A. Mehrdad, Y. Rotella, The subterranean 2.0 cipher suite, accessed: 2019-03-29 (March 29, 2019).